

Ans1

Operating system is a system-level software layer that controls hardware and provides services to user applications. It is like a manager between user programs and machine resources. The first objective of OS is convenience, meaning users and programmers should not worry about hardware-level complexity. The second objective is efficient utilization of CPU, memory, and I/O so that throughput and responsiveness remain high.

It performs process scheduling, memory allocation, file and device handling, permission control, and system security. It also offers abstractions like files, virtual memory, and process APIs which make application development practical. Therefore, OS is both a control program and a resource allocator.

Ans2

A process is an independent executing program with separate address space and resources. If one process crashes, others are usually protected due to isolation. A thread is a smaller execution path inside a process and shares memory and resources of that process.

Threads are lightweight because creating/switching threads requires less overhead than processes. Also communication between threads is faster because they share address space directly. Example: in a browser, UI thread handles interaction, network thread fetches data, and rendering thread paints content, all in parallel under one process.

Ans3

CPU scheduling decides which process from ready queue should run next. It is necessary in multiprogramming because many processes are ready but only one can use CPU core at a time. FCFS is non-preemptive and very simple. It follows arrival order, which is fair in queue sense but often inefficient. A long job at front delays smaller jobs and increases waiting time (convoy effect). Round Robin is preemptive and assigns each process a fixed quantum. This improves fairness and response time because no process can hold CPU too long. But RR has context-switch overhead, especially when quantum is too small. In practice, RR is preferred for interactive systems while FCFS may suit simple batch workloads.

Ans4

Deadlock is a condition in which processes are blocked forever because each is waiting for a resource held by another process. Coffman conditions explain when deadlock can occur:

1. Mutual exclusion: resource cannot be shared.
2. Hold and wait: process keeps one resource while asking another.

3. No preemption: resource cannot be forcefully taken.

4. Circular wait: chain of processes waiting for each other.

If all four happen together, deadlock is possible. Example: P1 holds printer and wants scanner; P2 holds scanner and wants printer. Neither proceeds. OS can handle deadlocks through prevention, avoidance, or detection/recovery methods.